

EXPERIMENT 6 – TIME SERIES ANALYSIS OF WEBSITE TRAFFIC

AIM

To analyze website traffic time series data to identify trends, seasonality and anomalies.

The manual requires website_traffic.csv but does not include the actual CSV/output. This completed version creates a reproducible sample daily dataset and applies the same methodology specified in the manual.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

np.random.seed(42)
dates = pd.date_range("2026-01-01", periods=90, freq="D")
trend = np.linspace(1000,1450,90)
seasonal = 120*np.sin(2*np.pi*np.arange(90)/7)
noise = np.random.normal(0,35,90)

data = pd.DataFrame({
    "Date":dates,
    "Page_Views":trend+seasonal+noise,
    "Unique_Visitors":trend*0.55+seasonal*0.35+np.random.normal(0,20,90),
    "Bounce_Rate":55+4*np.sin(2*np.pi*np.arange(90)/7)+np.random.normal(0,1.5,90)
})
data.loc[60,"Page_Views"] += 650
data.set_index("Date", inplace=True)
data.interpolate(inplace=True)

decomposition = seasonal_decompose(data["Page_Views"], model="additive", period=7)
decomposition.plot()
plt.show()

data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
data[["Page_Views","Page_Views_MA"]].plot()
plt.title("Page Views with Moving Average")
plt.show()

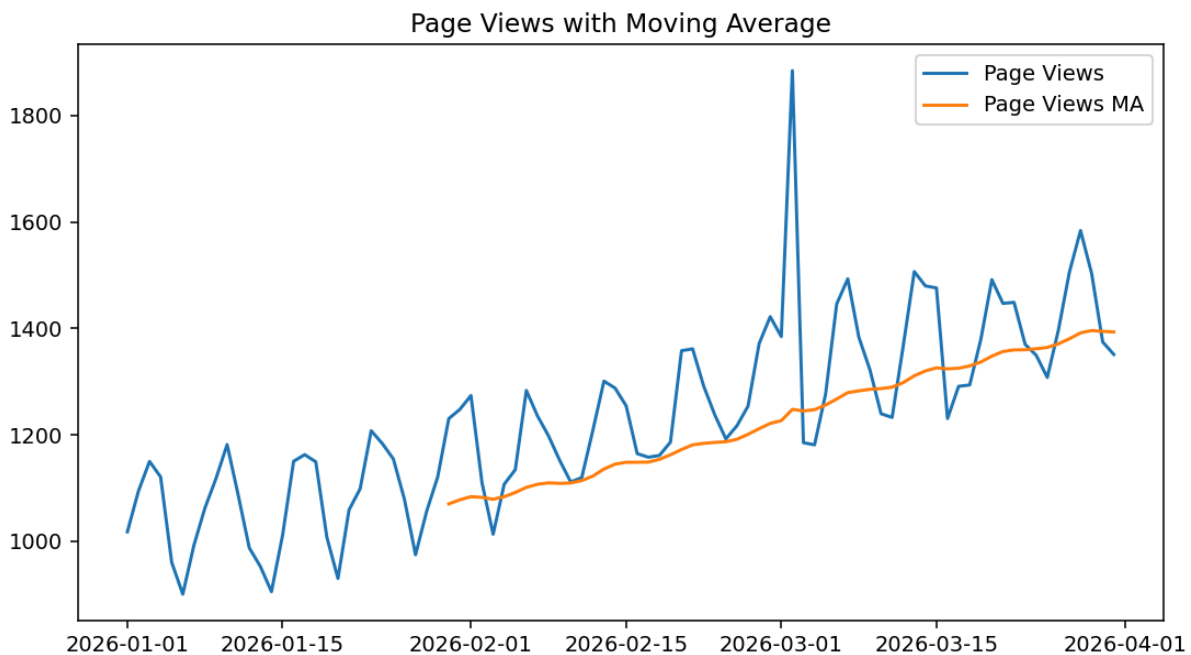
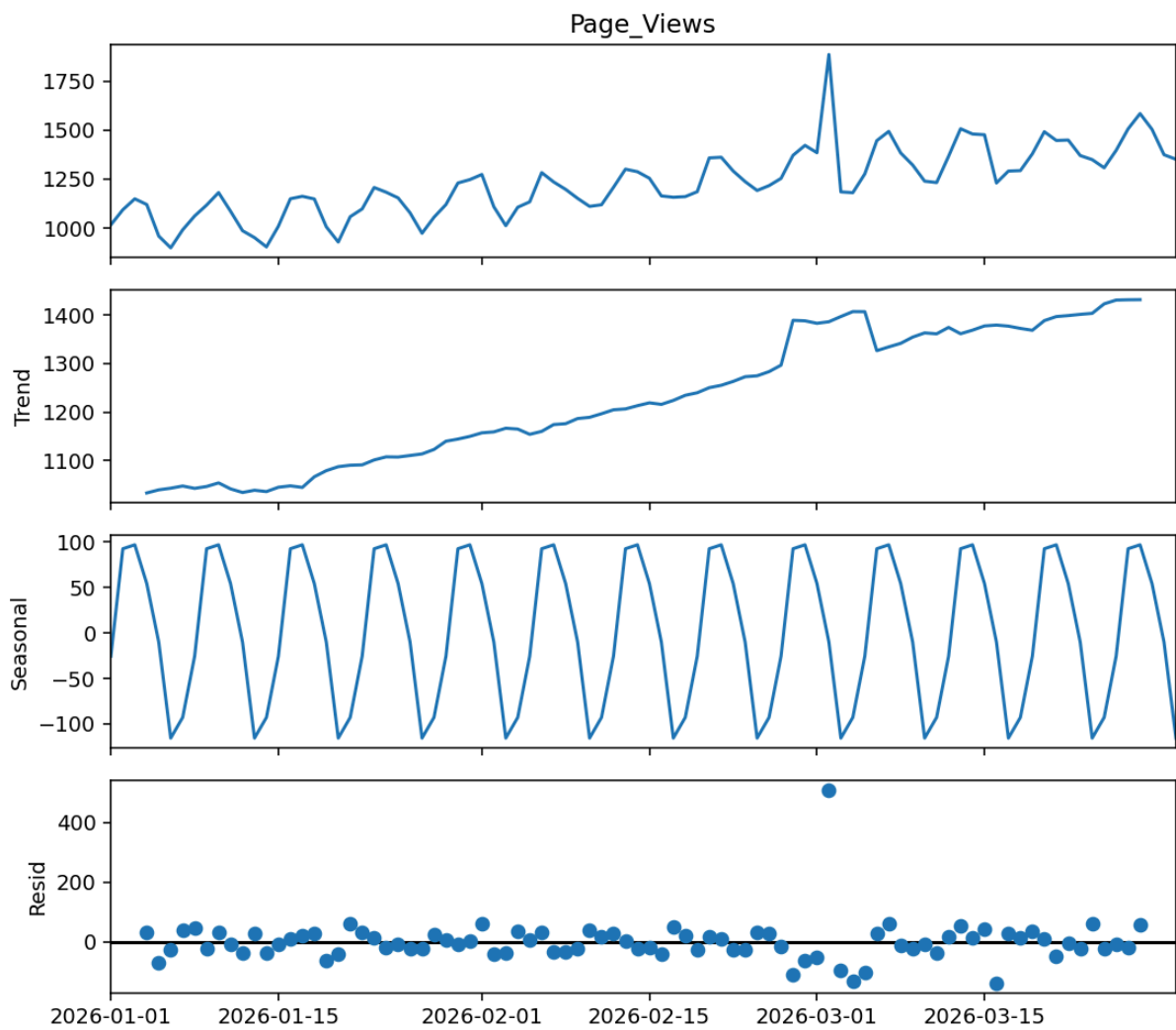
data["Page_Views_Z"] = (data["Page_Views"]-data["Page_Views"].mean())/data["Page_Views"].std()
anomalies = data[data["Page_Views_Z"].abs(>3)]
plt.plot(data.index,data["Page_Views"],label="Page Views")
plt.scatter(anomalies.index,anomalies["Page_Views"],color="red",label="Anomalies")
plt.legend()
plt.show()

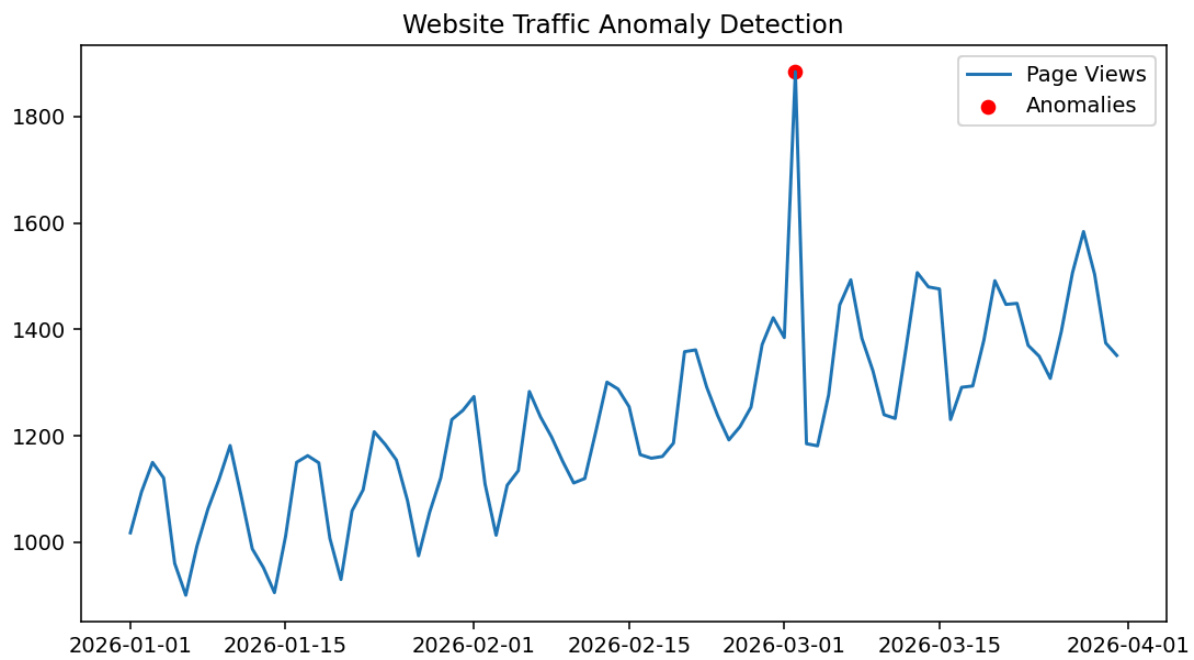
print("Number of anomalies:",len(anomalies))
print(anomalies[["Page_Views","Page_Views_Z"]])
```

OUTPUT

Number of anomalies: 1

	Page_Views	Page_Views_Z
2026-03-02	1884.533639	3.786145





Result

The website traffic time series was analyzed successfully using decomposition, moving averages and Z-score anomaly detection.